Design Track

Interview Preparation Series • Professional Edition

System Design Problems — Advanced

Compiled with LaTeX • June 17, 2026

harsha.somisetty01@gmail.com

In This Edition

Four fully worked FAANG-style system-design solutions. Each covers requirements, capacity math, API, data model, architecture, deep dives, scaling, and trade-offs.

1 1. Design a Video Streaming Service (YouTube / Netflix)

1.1 1. Requirements

Clarifying questions to ask:

- Are we designing upload + playback, or playback-only?
- Live streaming, or VOD (video on demand), or both?
- What's the target geography — global CDN or single-region?
- Should we design the recommendation engine, or just the data hooks?
- Premium / DRM requirements?

Functional Requirements:

- Users can upload videos (any resolution, up to ~10 GB raw).
- Platform transcodes video to multiple resolutions (240p, 360p, 720p, 1080p, 4K).
- Users can stream videos with adaptive bitrate (quality switches based on bandwidth).
- Search, browse, and play videos; like, comment, subscribe.
- Recommendation feed on homepage.

Non-Functional Requirements:

- Availability: 99.99% (< 53 min/yr downtime).
- Low latency playback start: < 2 s p99.
- Upload should not block playback — async processing pipeline.
- Globally distributed; content served from CDN edge nodes.
- Durability: 11 nines (video assets are irreplaceable for creators).
- Scale: YouTube-tier — 500 hours of video uploaded per minute; 1B+ DAU.

1.2 2. Capacity Estimation

Users & traffic:

```python
DAU             = 1 billion
Avg watch time  = 30 min/day
Videos uploaded = 500 hours/min = 30,000 hours/min raw footage
```

Read QPS (playback requests):

```
1B users × 5 video views/day = 5B views/day
5B / 86,400 s ≈ 58,000 views/sec  →  ~60K QPS peak
```

Write QPS (uploads):

```
500 hours/min ÷ 60 = ~8.3 hours/sec raw upload
~100 concurrent upload streams sustained at any time (most uploads are large, slow)
```

Storage (per year):

```
Raw:         500 hr/min × 60 min/hr × 24 hr/day × 365 days = 262,800,000 hr/yr
             Avg 1 hr video raw ≈ 2 GB → 525 PB/yr raw
Transcoded:  ~5 resolutions × 0.4× compression ratio each = ~2× raw
             Total ≈ 1 EB/yr  (YouTube stores ~1 EB today)
```

Bandwidth (outbound CDN):

```
60K concurrent streams × avg 3 Mbps (720p) = 180 Gbps
Peak (2× avg)                              = 360 Gbps served by CDN PoPs
```

Cost hint: At $0.01/GB CDN egress, 180 Gbps × 3600 s × 24 h = ~$1.5 M/day — hence CDN caching is existential.

1.3 3. API Design

Python

```python
# Upload
POST   /v1/videos/initiate-upload
       Body: { title, description, tags, category }
       Returns: { upload_id, upload_url (signed S3) }

PUT    /v1/videos/upload/{upload_id}/chunk
       Headers: Content-Range, Content-Length
       Body: <binary chunk>

POST   /v1/videos/upload/{upload_id}/complete
       Returns: { video_id, status: "processing" }

# Playback
GET    /v1/videos/{video_id}
       Returns: { metadata, manifest_url (HLS/DASH), thumbnail_url }

GET    /v1/videos/{video_id}/manifest.m3u8   (served by CDN)

# Discovery
GET    /v1/feed?user_id=&page_token=
GET    /v1/search?q=&sort=relevance&page=
GET    /v1/videos/{video_id}/recommendations

# Engagement
POST   /v1/videos/{video_id}/views      (fire-and-forget)
POST   /v1/videos/{video_id}/likes
POST   /v1/videos/{video_id}/comments
```

1.4 4. Data Model

Metadata DB — PostgreSQL (sharded) or Spanner (global):

SQL

```sql
videos (
  video_id        UUID PRIMARY KEY,
  creator_id      BIGINT,
  title           TEXT,
  description     TEXT,
  status          ENUM('uploading','processing','ready','failed'),
  duration_s      INT,
  raw_s3_key      TEXT,
  created_at      TIMESTAMPTZ,
```

```
  view_count      BIGINT,     -- denormalized, updated async
  like_count      BIGINT
)

video_renditions (
  video_id        UUID,
  resolution      TEXT,       -- '1080p', '720p', ...
  codec           TEXT,       -- 'h264', 'h265', 'av1'
  manifest_key    TEXT,       -- S3 key for HLS/DASH manifest
  segment_prefix  TEXT,       -- S3 prefix for .ts segments
  size_bytes      BIGINT
)

channels (channel_id, owner_user_id, subscriber_count, ...)
subscriptions (subscriber_id, channel_id, created_at)
comments (comment_id, video_id, user_id, body, created_at)
```

Why PostgreSQL/Spanner? Transactional metadata (status updates, renditions) needs ACID. Spanner if truly global consistency is required; sharded Postgres otherwise.

Counters — Redis or dedicated counter service:

- `VIEW:{video_id}` → HyperLogLog for approximate unique views; batched flush to Postgres.

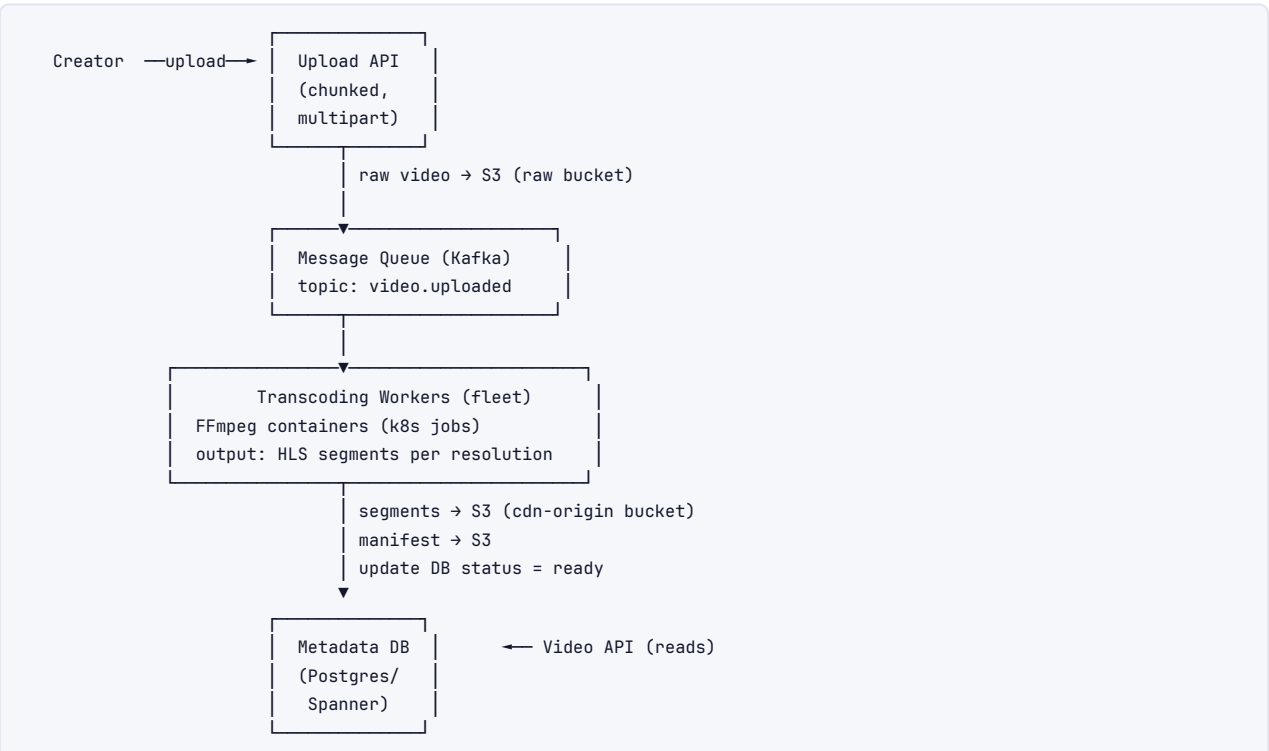
Search — Elasticsearch:

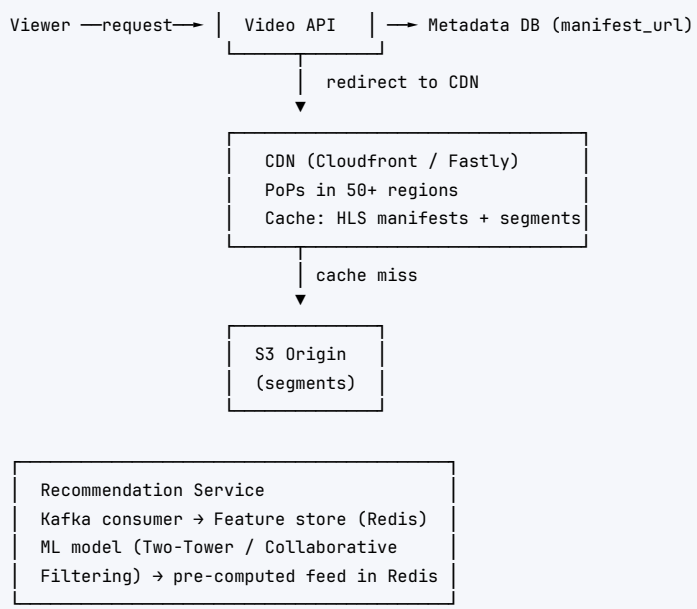
- Index: `{ video_id, title, tags, transcript_excerpt, view_count, created_at }`.
- Full-text + vector embedding for semantic search.

Blob Storage — S3-compatible object store (AWS S3 / GCS):

- Raw uploads in one bucket; transcoded segments in another bucket per region.
- Lifecycle policy: raw deleted after 30 days if status=ready.

1.5 5. High-Level Architecture





1.6 6. Deep Dives

6.1 Upload Pipeline & Transcoding

Chunked Upload:

1. Client calls `/initiate-upload` → receives a pre-signed S3 multipart upload URL.
2. Client uploads 5–10 MB chunks in parallel (S3 multipart supports up to 10K parts).
3. Client calls `/complete` → API assembles the multipart object in S3.
4. API publishes `{ video_id, s3_key }` to Kafka topic `video.uploaded`.

Transcoding Workers:

- Kubernetes Jobs pulled from Kafka; each worker pulls raw video, runs FFmpeg.
- **Parallel transcoding:** one worker per resolution, or one worker per video with all resolutions in parallel child processes.
- Output: HLS (`.m3u8` master manifest + `.ts` segment files per resolution, default 6 s segments).
- Also output a DASH `MPD` manifest for Chrome/Android compatibility.
- Store thumbnails (extracted at T=5s, 30s, 60s) in S3.
- Update `video_renditions` table and set `status = 'ready'`.

Resolution ladder (YouTube standard):

```
2160p (4K)   → bitrate ~20 Mbps
1080p        → ~8 Mbps
720p         → ~5 Mbps
480p         → ~2.5 Mbps
360p         → ~1 Mbps
240p         → ~0.5 Mbps
```

6.2 CDN Distribution

- S3 origin sits behind CDN (e.g., CloudFront with 400+ PoPs).
- HLS segments are tiny (< 1 MB each, 6 s) — extremely cache-friendly.
- **Cache TTL:** segments are immutable → TTL = ∞ (or 30 days). Manifests refreshed every 60 s.
- **Geo-routing:** DNS returns nearest PoP IP via Anycast/Route 53 Latency Routing.

- **Cache hit ratio target:** > 95% for popular videos. Long-tail videos miss to origin (cheap S3 GET).

6.3 Adaptive Bitrate Streaming --- HLS vs DASH

Feature	HLS (Apple)	DASH (MPEG)
Standard	Apple proprietary → RFC 8216	MPEG open standard
Container	MPEG-TS or fMP4	fMP4 (CMAF)
iOS/Safari	Native support	Requires MSE (not on older iOS)
Android/Chrome	Via MSE	Native support
DRM	FairPlay	Widevine / PlayReady
Latency (live)	6–30 s (LLHLS: 1–3 s)	2–8 s (CMAF chunked)
Adoption	YouTube, Netflix (backup)	Netflix (primary), Disney+

Key decision: Serve both HLS and DASH manifests from the same fMP4 segments (CMAF). This halves storage vs maintaining two separate segment sets.

6.4 Storage Tiers

```
Hot   (< 7 days old)   → S3 Standard          (fast GET, higher cost)
Warm  (7–90 days)      → S3 Infrequent Access (30% cheaper)
Cold  (> 90 days, < 1M views)  → S3 Glacier Instant Retrieval
Archive (> 1yr, < 10K views)   → S3 Glacier Deep Archive (pennies/GB)
```

6.5 Recommendations (brief)

- **Candidate generation:** Two-Tower model; user embedding × video embedding → top-K ANN (FAISS / ScaNN).
- **Ranking:** LightGBM or DNN re-ranker on features: CTR, watch time, freshness, diversity.
- **Serving:** Pre-compute homepage feed into Redis (TTL 30 min); real-time session signals update weights.
- **Cold start:** New videos bootstrapped with category/tag-based rules.

1.7 7. Bottlenecks & Scaling

Bottleneck	Mitigation
Transcoding backlog	Auto-scale k8s transcoding pods on Kafka consumer lag metric
Metadata DB hot rows (view counts)	Counter service with Redis; async batch-write to DB
CDN cache misses for new videos	Pre-warm CDN by pushing manifest URL to CDN after transcoding
S3 request throttling	Randomize S3 key prefixes (avoid date-prefix hotspot)
Recommendation latency	Pre-compute + cache feeds; async refresh
Single-region failure	Multi-region S3 replication; CDN absorbs reads; DB cross-region replica

1.8 8. Trade-offs Summary

Decision	Option A	Option B	**Choice**
Transcoding: per-video vs sharded	Single big VM per video	K8s pod per resolution	**K8s pods** (parallelism, cost)
Metadata DB	MySQL sharded	Spanner / CockroachDB	**Spanner** if global consistency needed
Streaming format	HLS-only	HLS + DASH (CMAF)	**Both** (broadest device coverage)
Recommendation serving	Real-time inference	Pre-computed feed	**Pre-computed** (latency wins)
Storage tiering	All S3 Standard	Tiered (hot/warm/cold)	**Tiered** (10× cost reduction)

Interview takeaways:

- **The upload path and the playback path are entirely decoupled** — async Kafka pipeline.
- **CDN cache hit ratio is the single most important scaling lever** for a video platform.
- **CMAF lets you serve HLS and DASH from the same segments** — always mention this.
- Transcoding is CPU-bound; scale it independently with Kafka consumer-lag autoscaling.

2 2. Design a Ride-Hailing Service (Uber / Lyft)

2.1 1. Requirements

Clarifying questions:

- Real-time ride-matching, or scheduled rides too?
- Single city or global deployment?
- Do we need the driver payout/financial system?
- Surge pricing — dynamic or simple multipliers?
- Bike/scooter/autonomous vehicles, or just cars?

Functional Requirements:

- Riders request a ride from location A to B.
- System matches rider to nearby available driver in real time.
- Driver and rider track each other on a live map during the trip.
- Compute fare estimate upfront; charge on trip completion.
- Surge pricing during high-demand periods.
- Driver/rider rating after trip.

Non-Functional Requirements:

- Matching latency < 3 s p99 from rider request to driver offer sent.
- Location update ingestion: 1–5 s refresh from every active driver.
- Availability: 99.99% (5 min/month downtime is unacceptable during peak hours).
- Consistency: eventual for location data; strong for trip state (no double-booking a driver).
- Global scale: 5M+ concurrent drivers; 10M+ concurrent riders during peak.

2.2 2. Capacity Estimation

```python
Active drivers (peak global)   = 5 million
Location update interval       = 4 s
Location update QPS            = 5,000,000 / 4 = 1,250,000 writes/sec

Active riders requesting rides = 500,000 concurrent
Ride requests QPS              = 500,000 / 10s avg session = 50,000 QPS

Trips per day                  = 25 million (Uber ~20M/day)
Trip data per trip             = ~2 KB metadata + GPS trace ~50 KB
Storage/yr                     = 25M × 52 KB × 365 = ~475 TB/yr

Location store size (in-memory):
  5M drivers × (lat, lon, heading, timestamp, driver_id) = ~200 bytes each
  = 5M × 200 B = 1 GB — fits in a large Redis cluster
```

2.3 3. API Design

```python
# Rider
POST   /v1/rides/estimate
       Body: { pickup_lat, pickup_lon, dest_lat, dest_lon, ride_type }
       Returns: { eta_s, fare_estimate, surge_multiplier }

POST   /v1/rides/request
       Body: { pickup_lat, pickup_lon, dest_lat, dest_lon, ride_type }
       Returns: { ride_id, status: "searching" }

GET    /v1/rides/{ride_id}
       Returns: { status, driver_id, driver_eta_s, driver_location }

DELETE /v1/rides/{ride_id}      (cancel)

# Driver
PUT    /v1/drivers/{driver_id}/location
       Body: { lat, lon, heading, speed, timestamp }

PUT    /v1/drivers/{driver_id}/status
       Body: { status: "available" | "busy" | "offline" }

GET    /v1/drivers/{driver_id}/ride-offers   (long-poll or WebSocket push)
POST   /v1/ride-offers/{offer_id}/accept
POST   /v1/ride-offers/{offer_id}/reject

# Shared real-time (WebSocket)
WS /v1/ws/ride/{ride_id}   (bidirectional: location pings, status events)
```

2.4 4. Data Model

Drivers (PostgreSQL):

```sql
drivers (
  driver_id       BIGINT PRIMARY KEY,
  name            TEXT,
  vehicle_id      BIGINT,
```

```
  rating          NUMERIC(3,2),
  current_status  ENUM('available','busy','offline'),
  city_id         INT
)
```

Rides (PostgreSQL sharded by ride_id):

```SQL
rides (
  ride_id         UUID PRIMARY KEY,
  rider_id        BIGINT,
  driver_id       BIGINT,
  status          ENUM('requested','searching','driver_assigned','in_progress','completed','cancelled'),
  pickup_lat      DOUBLE,
  pickup_lon      DOUBLE,
  dest_lat        DOUBLE,
  dest_lon        DOUBLE,
  requested_at    TIMESTAMPTZ,
  matched_at      TIMESTAMPTZ,
  completed_at    TIMESTAMPTZ,
  fare_estimate   NUMERIC(10,2),
  final_fare      NUMERIC(10,2),
  surge_mult      NUMERIC(4,2)
)
```

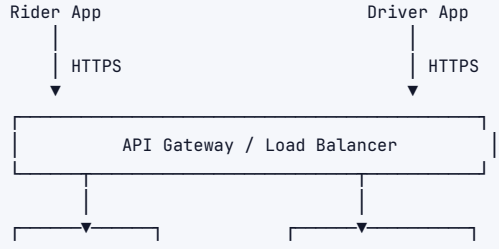
Driver Locations (Redis / in-memory geo store):

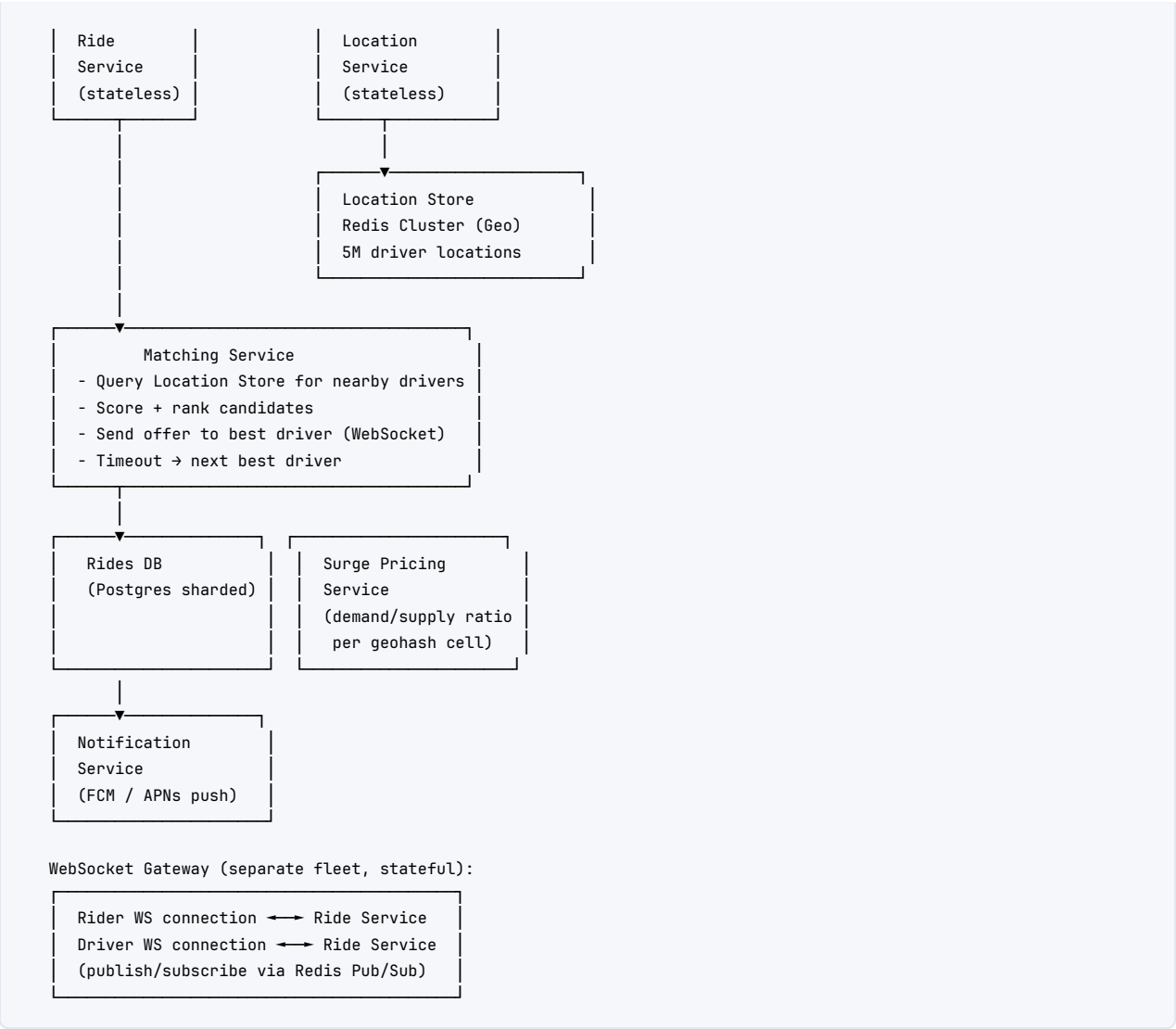
```Python
Key:   DRIVER_LOC:{driver_id}
Value: { lat, lon, heading, updated_at }
TTL:   30 s (stale if driver stops sending heartbeats)

Geo index: Redis GEOADD city:{city_id}:drivers {lon} {lat} {driver_id}
           Redis GEORADIUS for nearest-N lookup
```

Trip GPS traces (Cassandra or TimescaleDB):

```
trip_locations (
  ride_id     UUID,
  ts          TIMESTAMPTZ,
  lat         DOUBLE,
  lon         DOUBLE,
  PRIMARY KEY (ride_id, ts)
) — append-only, time-series access pattern
```

2.5 5. High-Level Architecture





2.6 6. Deep Dives

6.1 Geospatial Indexing --- Geohash vs Quadtree vs S2

Feature	Geohash	Quadtree	Google S2
Structure	Base-32 string, hierarchical	Tree of rectangular cells	Hilbert-curve cells on sphere
Precision control	String length (1–12 chars)	Tree depth	Level (0–30)
Query type	String prefix match	Tree traversal	Cell union + covering
Edge distortion	Yes (poles, antimeridian)	Yes (aspect ratio varies)	Minimal (spherical geometry)
Neighbor lookup	8 neighbors (manual computation)	Tree sibling traversal	S2::GetEdgeNeighbors
Used by	MongoDB, Elasticsearch, Redis	Older game engines	Uber (H3), Google Maps, Foursquare
Complexity	Simple	Moderate	Higher (library dependency)

Key decision: Use Geohash (precision level 6 ≈ 1.2 km cells) for driver lookup.

› Each driver is stored in Redis with `GEOADD` (which internally uses Geohash).

- On ride request: `GEORADIUS pickup_lat pickup_lon 3km` → list of driver IDs → filter by `status=available`.
- **Why not quadtree?** Redis GEODATA gives us geohash for free; quadtree requires custom in-memory structure.
- **Why not S2?** Overkill for matching; S2 shines in complex polygon coverage (e.g., Uber H3 for analytics).

6.2 Driver-Rider Matching

Algorithm:

1. `GEORADIUS` call returns N available drivers within 3 km (expand to 5 km if < 3 results).
2. Score each driver:

```python
score = w1 × (1 / ETA_to_pickup) + w2 × driver_rating + w3 × acceptance_rate
```

3. Send offer to top-scored driver. Wait 10 s.
4. If rejected/timeout → send to next driver. Max 5 tries before returning "no drivers available."
5. On acceptance → atomically set `rides.driver_id`, `rides.status = driver_assigned`, `drivers.current_status = busy` (Postgres transaction).

Preventing double-assignment: Use Postgres `SELECT FOR UPDATE` or Redis distributed lock (`SET driver:{id}:lock NX EX 15`) before assigning.

6.3 Real-Time Location Updates

- Drivers send GPS ping every 4 s via HTTP/2 (or WebSocket for persistent connection).
- **Location Service** writes to Redis `GEOADD` (O(log N)) — 1.25M writes/sec requires Redis Cluster with 10–20 shards.
- **Sharding:** partition by `city_id` or geohash prefix (level 2 ≈ 630 km cells → ~100 world cells → manageable shards).

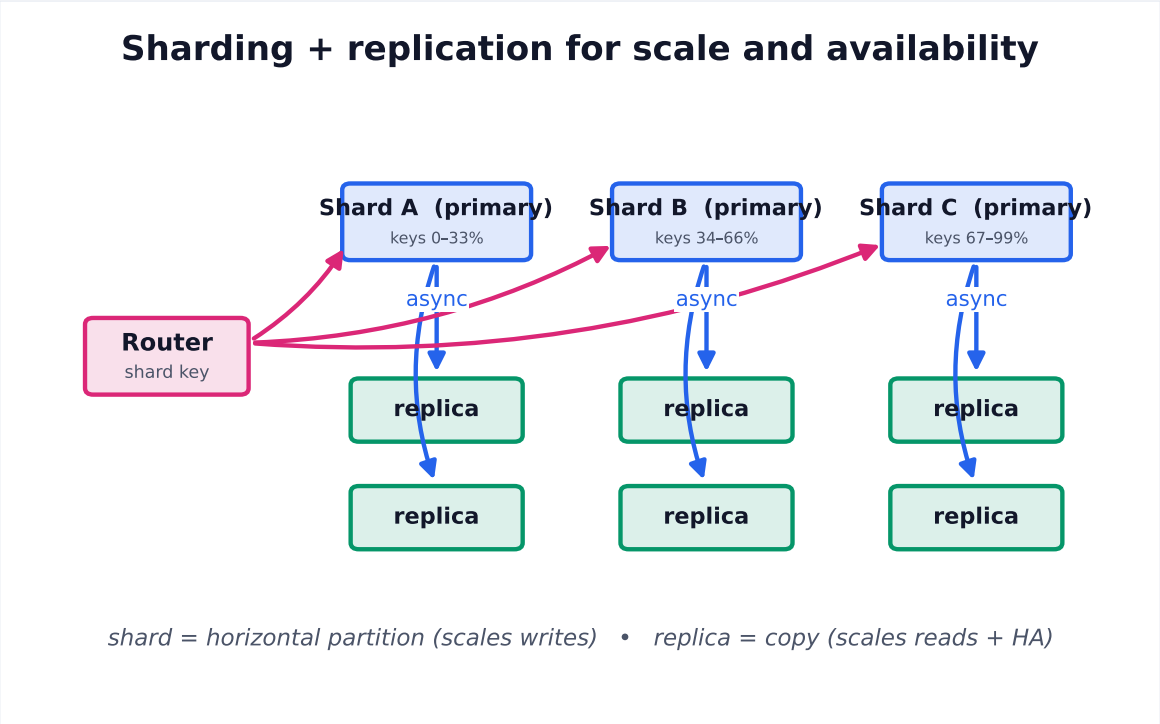


Figure 1. Sharding partitions data by key to scale writes across nodes; replication copies each shard for read scaling and failover. Most large datastores combine both.

- Driver location also published to **Kafka** topic `driver.location` for:
  - Live map updates pushed to rider via WebSocket.
  - Surge pricing calculation.
  - ETA recalculation.
  - Trip trace storage (Cassandra consumer).

6.4 Surge Pricing

```python
demand_cells[geohash6] = count of ride requests in last 5 min
supply_cells[geohash6] = count of available drivers in cell

surge_multiplier = max(1.0, demand / (supply × target_utilization_factor))
```

- Computed every 30 s by Surge Service consuming Kafka streams.
- Stored in Redis: `SURGE:{geohash6}` → multiplier (float).
- Ride estimate reads from Redis before quoting price.
- **UI UX:** show surge visually (heatmap) and require explicit rider confirmation if multiplier > 1.5×.

6.5 Trip Lifecycle --- State Machine

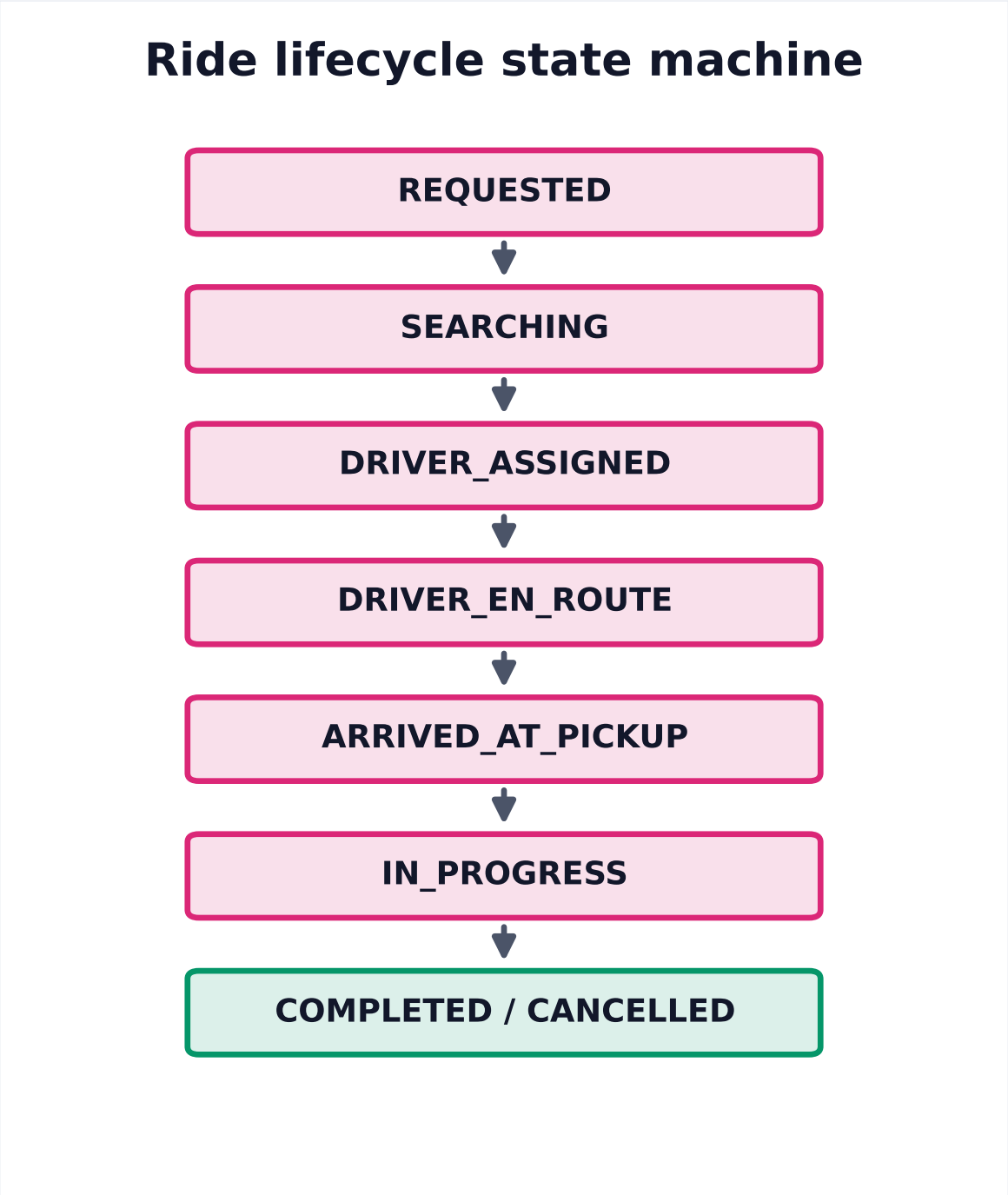


Figure 2. Modeling the ride as an explicit state machine makes valid transitions (and the events that trigger them) unambiguous — the backbone of the matching and trip services.

State transitions are persisted in Postgres with timestamps. Events published to Kafka on each transition. Kafka consumers handle: notifications, billing trigger (on COMPLETED), analytics.

6.6 ETA Calculation

- **Routing engine:** OSRM or Google Maps Platform API call on driver assignment.
- **Real-time recalculation:** every 30 s while driver is en route, re-query routing engine with current driver location.
- **Pre-computed ETA surfaces:** city road graph stored in-memory (contraction hierarchies) for sub-100ms routing. Uber uses their own in-house routing (H3-based graph).

2.7 7. Bottlenecks & Scaling

Bottleneck	Mitigation
Location write storm	Redis Cluster, shard by city/geohash; accept at-most-once semantics
Matching hot cities (NYC peak)	City-level matching service instances; limit to local Redis shard
Rides DB write contention	Shard by `city_id`; use UUIDs to avoid hot auto-increment keys
WebSocket connection scaling	Stateful WS servers behind L4 LB; use Redis Pub/Sub as broker
Driver double-booking	Redis NX lock on `driver_id` during match; release on timeout
Surge pricing staleness	Short TTL (30 s) + frequent recalculation; eventual consistency OK

2.8 8. Trade-offs Summary

Decision	Option A	Option B	**Choice**
Geospatial index	Geohash (Redis GEODATA)	Custom Quadtree	**Geohash (Redis)**
Driver location transport	HTTP polling (4 s)	Persistent WebSocket	**HTTP/2** (simpler, scales well)
Matching concurrency control	Postgres FOR UPDATE	Redis distributed lock	**Redis lock** (lower latency)
Location store	Redis in-memory	PostGIS	**Redis** (1M+ writes/sec)
Trip state persistence	Postgres	Cassandra	**Postgres** (ACID for state machines)
Routing engine	Google Maps API	Self-hosted OSRM	**OSRM** at scale (cost)

Interview takeaways:

- **The geospatial indexing approach is the core of this design** — be ready to explain Geohash precision levels.
- **Driver location is write-heavy, trip state is consistency-critical** — use different stores for each.
- **The matching loop is an iterative fan-out** — must handle timeouts and fallbacks gracefully.
- Surge pricing uses supply/demand ratio per geohash cell — a simple but powerful formula.

3 3. Design a Distributed Cache (like Redis/Memcached at Scale)

3.1 1. Requirements

Clarifying questions:

- Key-value only, or also sorted sets, pub/sub, streams?
- What are the SLAs on read/write latency?
- Strong consistency needed, or eventual?
- Target use case: session cache, DB cache, rate limiter, leaderboard?
- What's the expected working set size (total hot data)?

Functional Requirements:

- `GET key`, `SET key value [TTL]`, `DEL key`, `MGET`, `MSET`.

- TTL-based expiration.
- LRU eviction when memory is full.
- Replication for high availability.
- Horizontal scaling via consistent hashing.
- Pub/Sub (optional advanced feature).

Non-Functional Requirements:

- Read latency < 1 ms p99.
- Write latency < 2 ms p99.
- Availability: 99.99% (cache miss falls through to DB, but should be rare).
- No data loss for critical caches (optional persistence via WAL/RDB).
- Scale to terabytes of cached data across a cluster.

3.2 2. Capacity Estimation

```
Cached data size (working set)   = 10 TB
Node RAM (cache node)            = 256 GB
Nodes needed                     = 10 TB / 256 GB ≈ 40 cache nodes

Read QPS                         = 1,000,000 reads/sec (1M QPS)
Write QPS                        = 100,000 writes/sec (10% of reads)

Per-node QPS (40 nodes)          = 25,000 reads + 2,500 writes per node
Single Redis node can handle     ≈ 100K–200K ops/sec → 40 nodes has 4–8× headroom

Network bandwidth per node:
  25K reads × avg 1 KB value     = 25 MB/s per node  (well within 10 GbE NIC)

Memory per key overhead (Redis): ~70 bytes metadata + value size
  1B keys × 70 B overhead        = 70 GB overhead in cluster (significant at scale)
```

3.3 3. API Design

Client-facing (wire protocol — Redis-compatible RESP):

```
GET  <key>                   → value | (nil)
SET  <key> <value> [EX secs] → OK
DEL  <key> [key ...]         → (integer) count deleted
MGET <key> [key ...]         → array of values
MSET <key> <val> [key val]   → OK
TTL  <key>                   → seconds remaining
INCR / DECR <key>            → new integer value
EXPIRE <key> <secs>          → 1 (success) | 0 (no such key)
```

Admin / Internal API:

```
CLUSTER ADDNODE <host:port>  → join node to cluster
CLUSTER FAILOVER             → trigger manual leader promotion
INFO                         → stats (memory, QPS, hit_rate, evictions)
BGSAVE                       → async RDB snapshot
```

3.4 4. Data Model

In-memory data structures:

```
Hash table: key → { value, type, TTL_expiry_ts, LRU_clock }

Value types:
  String → raw bytes (up to 512 MB)
  List   → doubly-linked list (small) or ziplist (< 128 elements)
```

```
Hash   → hash table or ziplist
Set    → hash table or intset
ZSet   → skip list + hash table
```

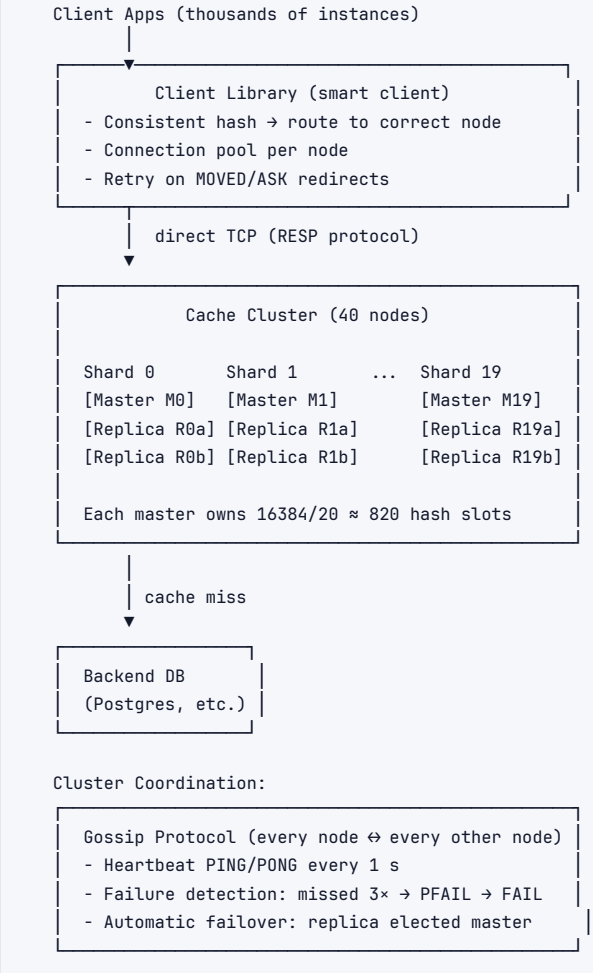
Persistence (optional):

```
RDB snapshot: point-in-time binary dump, every N seconds or M writes
AOF (Append-Only File): write-ahead log, fsync policy:
  - always  → 1 write per command, safest, slowest
  - everysec → fsync every 1s, at most 1s of data loss
  - never   → OS decides, fastest, most data loss risk
```

Cluster topology metadata:

```
Cluster config (stored in every node):
  slot_assignments[0..16383] → node_id   (Redis uses 16384 hash slots)
  node_registry: { node_id, host, port, role: master|replica, slots }
```

3.5 5. High-Level Architecture



3.6 6. Deep Dives

6.1 Consistent Hashing

Problem: Naive `hash(key) % N` rehashes ALL keys when N changes (add/remove node).

Solution — Consistent Hash Ring:

```
Virtual ring: 0 to 2^32
Each node placed at multiple positions (virtual nodes / vnodes)
  - 150 vnodes per physical node → even distribution
  - Rebalancing a new node: only transfer 1/N of keys (not all)

key → hash(key) → position on ring → clockwise next node = owner

                  0
                 /|\
           M3 ←  |  → M0
          /      |      \
        M2       |       M1
          \              /
            ————————————
                2^32
```

Key decision: 150–200 virtual nodes per physical node balances load even with heterogeneous node sizes (weight vnodes by RAM). With 40 nodes × 150 vnodes = 6,000 points on ring.

6.2 Replication

- Each shard: 1 master + 2 replicas (RF=3).
- **Async replication** (default Redis): master returns OK to client, replicates to replicas asynchronously. Risk: up to N ms of data loss on crash.
- **Semi-sync (WAIT command):** `WAIT 1 100` — wait for 1 replica to ack within 100ms. Better durability at slight latency cost.
- Replicas serve **reads** to scale read-heavy workloads (with possible stale reads).
- On master failure: replica with lowest replication lag is elected master (Raft-adjacent gossip vote).

6.3 Eviction Policies (LRU and Beyond)

Policy	Description	Use case
noeviction	Return error when memory full	Primary DB (never evict)
allkeys-lru	Evict globally least-recently-used key	General cache
volatile-lru	Evict LRU only among keys with TTL set	Mixed TTL/no-TTL data
allkeys-lfu	Evict least-frequently-used (Redis 4.0+)	Skewed access patterns
volatile-ttl	Evict key with shortest remaining TTL	Time-sensitive caching
allkeys-random	Evict random key	Uniform access, simplest

Approximate LRU (Redis implementation):

- Full LRU requires O(N) scan — too slow.
- Redis samples 5 (configurable) random keys, evicts the least-recently-used among them.
- At sample=10, approximates true LRU with < 1% error in most workloads.

Key decision: Use `allkeys-lfu` for a general-purpose cache — LFU outperforms LRU when there are periodic scans that pollute the LRU queue with cold data.

6.4 Cache Invalidation Strategies

Strategy	Description	Pros	Cons
TTL expiry	Set TTL on write; auto-expire	Simple	Stale window = TTL duration

Strategy	Description	Pros	Cons
Write-through	Write to cache + DB simultaneously	Always consistent	Write latency increased
Write-behind (async)	Write to cache, async flush to DB	Low write latency	Data loss risk
Cache-aside	App reads cache; miss → read DB → populate cache	DB is source of truth	Cache miss latency spike
Read-through	Cache library fetches from DB on miss automatically	Transparent to app	Library complexity
Event-driven invalidation	DB change → CDC event → DEL from cache	Real-time consistency	Infrastructure complexity

Key decision: TTL + event-driven invalidation (CDC via Debezium/Kafka). TTL covers eventual consistency; CDC invalidates on DB write for critical data.

6.5 Hot Key Problem

Problem: A single key (e.g., a celebrity's profile `user:12345`) gets 100K QPS — overwhelming the one shard that owns it.

Solutions:

1. **Key replication / local cache:** Client-side in-process cache (LRU, 100 MB per app server) for top-N hot keys. App caches key for 1 s locally.
2. **Scatter with suffix:** Store key as `user:12345:0` through `user:12345:9` (10 shards); reads pick random suffix, writes update all. Works for read-heavy hot keys.
3. **Read replicas:** Route reads to any of the replicas for that shard.
4. **Detect & alert:** Monitor per-key QPS in the client library; auto-promote to local cache.

6.6 Cache Stampede (Thundering Herd)

Problem: Popular key expires → 10K concurrent requests all miss → all hit DB simultaneously → DB overloaded.

Solutions:

1. **Probabilistic early expiration (PER):** Recompute cache key slightly before TTL expires (during window `TTL - random(0, β × log(compute_time))`). Only one client recomputes at a time; others still serve stale value.
2. **Mutex / lock:** First cache-miss acquires a distributed lock (Redis `SET lock:key NX EX 5`). Others wait or serve stale.
3. **Background refresh:** Separate async job refreshes popular keys before they expire.
4. **Stale-while-revalidate:** Serve stale immediately; trigger async refresh; update cache when done.

Key decision: Stale-while-revalidate for most use cases — zero extra latency for users; background refresh handles expiry.

6.7 Write Policies

```
Write-Through:   Client → Cache & DB (synchronous both)
                 + Consistent  - Higher write latency

Write-Back:      Client → Cache (immediate) → DB (async batch)
                 + Low latency  - Risk of data loss

Cache-Aside:     Client → DB directly; read-path populates cache
                 + DB authoritative  - Cold start cache misses
```

6.8 Node Failure Handling

- **Detection:** Gossip heartbeat; node marked `PFAIL` after missing 3 pings; cluster votes to mark `FAIL` (majority of masters agree).
- **Failover:** Replica with smallest replication offset (least lag) wins election; becomes new master for the slot range within ~1–3 s.
- **Client impact:** During failover window, affected slots return `CLUSTERDOWN` errors → client retries with backoff.
- **Split brain prevention:** Requires majority of masters alive to accept writes. With 20 masters, lose 10 → writes rejected (availability vs consistency trade-off).

3.7 7. Bottlenecks & Scaling

Bottleneck	Mitigation
Hot key (single shard 100K QPS)	Key fan-out / local in-process cache
Memory exhaustion	Eviction policy; monitor `used_memory_rss`; add nodes
Network bandwidth (large values)	Compress values at client (snappy/lz4); max value size policy
Failover latency (1–3 s)	Keepalive + health checks; pre-elect backup masters
Replication lag	`WAIT 1 100` for critical writes; monitor `repl_lag` metric
Cross-datacenter replication	Redis Cluster doesn't span DCs natively; use CRDT-based solutions or geo-replicas with eventual consistency

3.8 8. Trade-offs Summary

Decision	Option A	Option B	**Choice**
Sharding strategy	Consistent hashing	Range-based sharding	**Consistent hashing** (smooth rebalance)
Eviction	LRU	LFU	**LFU** (scan-resistant)
Cache invalidation	TTL only	TTL + CDC events	**TTL + CDC** (real-time for critical)
Write policy	Write-through	Cache-aside	**Cache-aside** (DB authoritative)
Hot key	Replica reads	Client-side local cache	**Local cache** (zero network)
Stampede protection	Mutex lock	Stale-while-revalidate	**SWR** (no latency penalty)
Persistence	RDB only	AOF everysec	**AOF everysec** (< 1s loss)

Interview takeaways:

- **Consistent hashing with virtual nodes is essential** — explain the 150-vnode trick for even distribution.
- **Hot key and cache stampede are the two most common follow-up questions** — have concrete solutions ready.
- **Redis uses approximate LRU (sampling)** — not true LRU. Know why.
- Cache-aside is the most common pattern; write-through for critical financial data.

4 4. Design a Web Crawler (Google-Scale)

4.1 1. Requirements

Clarifying questions:

- Full web crawl (all URLs), or focused crawl (specific domains/topics)?
- How fresh does the data need to be? (daily recrawl vs. weekly vs. on-change)
- Should we store raw HTML or parsed content?
- Do we need to handle JavaScript-rendered pages (dynamic content)?
- What's the target crawl rate and storage budget?

Functional Requirements:

- Start from a set of seed URLs; discover new URLs via link extraction.
- Download and store page content.
- Respect `robots.txt` and crawl-delay directives.
- Deduplicate URLs — don't crawl the same page twice (in a given window).
- Freshness — recrawl pages periodically based on update frequency.
- Output: raw HTML (and/or extracted text) + metadata per URL.

Non-Functional Requirements:

- Scale: crawl 10 billion pages (Common Crawl scale).
- Throughput: 1,000 pages/sec sustained (86M pages/day → 10B pages in ~116 days).
- Distributed: hundreds of crawler nodes.
- Politeness: max 1 req/sec per domain; honor robots.txt.
- Durability: don't re-crawl already-crawled URLs unless recrawl is due.
- Freshness: high-value pages (news) re-crawled hourly; static pages monthly.

4.2 2. Capacity Estimation

```
Target pages           = 10 billion
Crawl rate             = 1,000 pages/sec
Time to crawl all      = 10B / 1,000 = 10,000,000 s ≈ 116 days  (1 full crawl cycle)
Peak rate needed       = 3,000–5,000 pages/sec (to finish faster / handle crawl failures)

Avg page size          = 100 KB HTML
Storage (HTML only)    = 10B × 100 KB = 1 PB
Compressed (~5:1 gzip) = ~200 TB

Links extracted per page = avg 50 outgoing links
Total URL queue entries  = 10B × 50 = 500B (dedup reduces this dramatically)
Seen URL set (dedup)     = 10B URLs × ~100 bytes/URL = 1 TB (in memory: impractical)
                           Bloom filter: 10B items, 1% FP → ~12 GB (fits in RAM!)

DNS lookups:
  1,000 pages/sec; avg 50 unique domains/sec
  DNS cache (TTL=300s) covers most; ~5 uncached DNS lookups/sec
```

4.3 3. API Design

Internal service APIs (not user-facing):

Python

```python
# URL Frontier Service
POST   /frontier/enqueue
       Body: { url, priority, source_page, discovered_at }
```

```
        Returns: { queued: true | false (already seen) }

GET     /frontier/next?crawler_id=&batch_size=100
        Returns: [{ url, fetch_after_ts }]  (respects politeness delay)

POST    /frontier/done
        Body: { url, status_code, crawled_at, content_hash }

# Content Store
PUT     /content/{url_hash}
        Body: { raw_html, headers, crawled_at, content_hash }

GET     /content/{url_hash}

# Robots.txt Cache
GET     /robots?domain=example.com
        Returns: { allowed_paths, disallowed_paths, crawl_delay_s, sitemap_urls }

# Recrawl Scheduler
POST    /recrawl/schedule
        Body: { url, next_crawl_at, priority }
```

4.4 4. Data Model

URL Frontier (Priority Queue + Politeness Queue):

```python
frontier_urls (
  url_hash        BIGINT  PRIMARY KEY,    -- xxHash64 of normalized URL
  url             TEXT,
  priority        FLOAT,                  -- higher = crawl sooner
  domain          TEXT,
  next_fetch_ts   TIMESTAMPTZ,            -- earliest time to fetch (politeness)
  status          ENUM('pending','in_flight','done','error'),
  retry_count     INT DEFAULT 0
)
Implemented as: Apache Kafka topics per priority band (high/medium/low)
                + Redis ZSETs per domain for politeness scheduling
```

Crawled Pages (blob + metadata):

```
pages (
  url_hash        BIGINT PRIMARY KEY,
  url             TEXT,
  domain          TEXT,
  content_hash    BIGINT,        -- SimHash for near-duplicate detection
  crawled_at      TIMESTAMPTZ,
  http_status     INT,
  content_type    TEXT,
  raw_html_key    TEXT           -- S3 key for raw HTML blob
)
```

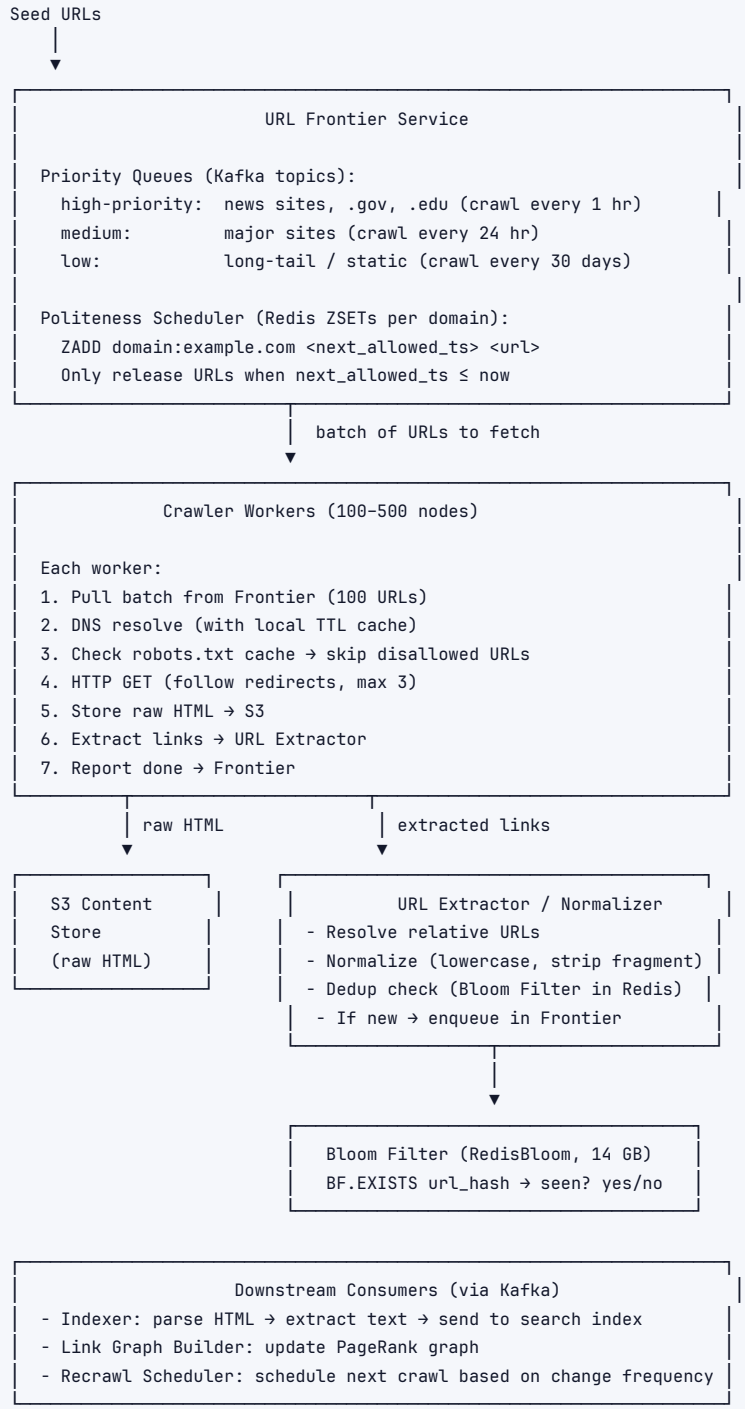
Seen URL Set (deduplication):

```
Bloom Filter (in-memory, distributed):
  - 10B URLs, 0.1% false positive rate → ~14.4 GB
  - Implementation: Redis BF.ADD / BF.EXISTS (RedisBloom module)
  - Fallback: persistent seen_urls table in Cassandra for exact check on BF hit
```

robots.txt Cache:

```
robots_cache (domain → RobotsTxt object, TTL=24h) in Redis
```

4.5 5. High-Level Architecture



4.6 6. Deep Dives

6.1 URL Frontier --- Priority + Politeness

Two-tier queue architecture (Mercator design):

```
Front-end queues (priority):      Back-end queues (politeness):
  Q_high  [news, trending]          Domain A: [url1, url3, url7]  → min 1s delay
  Q_med   [general web]             Domain B: [url2, url5]        → min 1s delay
  Q_low   [static, rarely updated]  Domain C: [url4, url6]        → min 5s delay

Selector: reads from front-end queues proportionally to priority.
          Routes each URL to its domain back-end queue.
          Back-end queue dispatcher waits for per-domain cooldown before releasing.
```

Priority assignment:

- PageRank estimate of source page.
- Domain authority (Alexa/Majestic rank).
- Time since last crawl vs. estimated update frequency.
- Explicit signals (sitemap priority, news feeds).

Implementation:

- Priority bands implemented as **Kafka topics** (`crawl.high`, `crawl.med`, `crawl.low`).
- Politeness tracking: **Redis sorted set** per domain — `ZADD domain:example.com {next_fetch_ts} {url}`. Poller does `ZRANGEBYSCORE domain:* -inf now LIMIT 10` every second.

6.2 URL Deduplication --- Bloom Filter vs Exact Set

Approach	Storage for 10B URLs	Lookup time	False positives	False negatives
Bloom Filter (1% FPR)	~12 GB	O(k) hashes	1% (skip new URL)	0%
Bloom Filter (0.1% FPR)	~14.4 GB	O(k) hashes	0.1%	0%
Hash Set (in-memory)	~1 TB (impractical)	O(1)	0%	0%
Cassandra exact store	~200 GB on disk	1–5 ms	0%	0%
Redis Set	~600 GB	O(1)	0%	0%

Key decision: Bloom Filter as first-pass dedup (RedisBloom, ~14 GB), with Cassandra exact-set for conflict resolution on BF hit (false positive check).

- BF says ”seen” → check Cassandra exact table (1% of traffic → Cassandra QPS manageable).
- BF says ”not seen” → guaranteed new URL → enqueue directly.

URL normalization before hashing:

```
1. Lowercase scheme and host
2. Remove default port (:80, :443)
3. Sort query parameters alphabetically
4. Remove URL fragment (#anchor)
5. Resolve relative paths (/../ → /)
6. Strip tracking params (utm_source, fbclid, etc.)
7. Canonicalize: prefer canonical link rel tag in HTML
```

6.3 DNS Resolution

Problem: 1,000 pages/sec from 1,000 different domains = 1,000 DNS lookups/sec. Public DNS resolvers (8.8.8.8) rate-limit at ~few hundred QPS.

Solutions:

1. **Local DNS cache per crawler node:** TTL-respecting LRU cache. Cache hit rate > 90% for popular domains.
2. **Dedicated DNS resolver cluster:** Internal recursive DNS servers (unbound / BIND), scaled horizontally. Query rate: ~50 uncached lookups/sec per node.
3. **Prefetch DNS:** When a new domain is discovered, pre-resolve asynchronously before it's scheduled for crawl.
4. **DNS blacklist:** Skip private IPs (SSRF protection: 10.0.0.0/8, 192.168.0.0/16, 127.0.0.1).

6.4 robots.txt Compliance

```
For domain example.com:
  1. Check Redis cache: GET robots:example.com
  2. Cache miss → fetch https://example.com/robots.txt
  3. Parse:
       User-agent: *
       Disallow: /private/
       Crawl-delay: 5
       Sitemap: https://example.com/sitemap.xml
  4. Store in Redis: SET robots:example.com <parsed_rules> EX 86400
  5. Before each URL fetch: check if path matches Disallow patterns

Politeness: max(1s_default, Crawl-delay_from_robots)
```

Gotchas:

- robots.txt itself may not exist (HTTP 404 → treat as "allow all").
- robots.txt may be too large (truncate at 512 KB).
- Different user-agent rules: use "Googlebot" or your crawler's name.

6.5 Trap Avoidance

Spider traps generate infinite URLs (e.g., `/calendar/next?date=...` → infinite future dates). Mitigation:

1. **URL depth limit:** max 10 hops from seed URL.
2. **Per-domain URL count limit:** max 100K URLs per domain (override for news sites).
3. **URL path pattern detection:** if same path template seen > 1,000 times, blacklist pattern.
4. **Cycle detection:** if URL hash appears in path history (seen in URL chain) → stop.
5. **Content hash dedup:** if page content SimHash is near-duplicate of already-crawled page → skip storing, but do extract links (might have new ones).

6.6 Near-Duplicate Detection (SimHash)

```
1. Extract tokens (words) from page HTML.
2. For each token, compute hash (64-bit).
3. For each bit position 0..63:
     if token_hash bit is 1 → add weight to column[bit]
     if token_hash bit is 0 → subtract weight
4. Final SimHash: bit[i] = 1 if column[i] > 0, else 0.

Two pages with SimHash Hamming distance ≤ 3 → near-duplicate.
Storage: 8 bytes per page × 10B pages = 80 GB → manageable in distributed store.
```

6.7 Distributed Coordination

Problem: 500 crawler nodes must not fetch the same URL concurrently.
Solution:

- URL Frontier is the single coordinator.
- Each URL assigned to one crawler node (round-robin or URL-hash based routing).

- Frontier marks URL `in_flight` on dispatch; reverts to `pending` if heartbeat missing for 60 s (dead crawler).
- No two nodes ever assigned the same URL from the frontier (frontier is authoritative).

Crawler node health:

- Periodic heartbeat to Frontier Service (every 10 s).
- Dead node's `in_flight` URLs requeued automatically after 60 s timeout.

6.8 Freshness & Recrawl Scheduling

Page type	Crawl frequency	Signal
News article (first 1h)	Every 15 min	Source is major news domain + recent publish
News article (1–24h)	Every 2 hr	Declining share velocity
Popular blog post	Daily	Sitemap `changefreq=daily`
Product page	Weekly	Sitemap `changefreq=weekly`
Static reference page	Monthly	Sitemap `changefreq=monthly` or inferred
Newly discovered URL	Once (ASAP)	Not yet crawled

Adaptive recrawl:

- Track `change_rate = content_hash_changes / recrawls` per URL.
- Increase frequency if page changes often; decrease if rarely changes.
- `next_crawl_delay = max(min_delay, last_crawl_interval × (1 - change_rate + 0.1))`

4.7 7. Bottlenecks & Scaling

Bottleneck	Mitigation
URL Frontier throughput	Partition Kafka topics by domain-hash; horizontal Frontier shards
Bloom filter accuracy drift	Use Counting Bloom Filter or periodic rebuild; monitor FP rate
DNS resolution rate	Dedicated internal DNS cluster + per-node LRU cache
S3 write throughput	Randomize S3 key prefix; use multipart upload for large HTML batches
robots.txt fetch overhead	Redis cache (TTL=24h); prefetch on domain discovery
Spider traps infinite loop	URL depth limit + per-domain URL count cap
Crawler node failure	Frontier requeues in_flight URLs after heartbeat timeout
Hot domains (Reddit, Twitter)	Per-domain rate limit enforced strictly; queue backs up, not domain

4.8 8. Trade-offs Summary

Decision	Option A	Option B	**Choice**
URL dedup	Exact hash set (Cassandra)	Bloom filter + exact fallback	**BF + exact fallback** (14 GB vs 200 GB)
URL frontier storage	Database (Postgres)	Kafka topics + Redis ZSETs	**Kafka + Redis** (throughput)

Decision	Option A	Option B	**Choice**
DNS	Public resolvers (8.8.8.8)	Internal recursive DNS cluster	**Internal DNS** (rate limit avoidance)
Content storage	HDFS	S3	**S3** (elastic, cheap, managed)
Near-duplicate detection	Full content MD5 comparison	SimHash (64-bit)	**SimHash** (O(1) compare, 8B/page)
JS rendering	Skip JS pages	Headless Chrome (Puppeteer)	**Skip first pass** (cost); headless for top-N sites
Crawl scheduling	Fixed interval	Adaptive (change-rate based)	**Adaptive** (better freshness/cost ratio)

Interview takeaways:

- **The URL Frontier is the heart of the crawler** — the two-tier priority + politeness design (Mercator) is the canonical answer.
- **Bloom filter for dedup at 10B scale** — know the math: 10B items, 0.1% FPR → 14.4 GB. This is the key insight.
- **Politeness is non-negotiable** — not just ethical; aggressive crawling gets IP-banned, defeating the crawler.
- **Distributed coordination:** URL Frontier as single source of truth prevents duplicate crawling across nodes.
- **Spider trap avoidance:** URL depth limit + per-domain URL count cap are the two simple, effective defenses.

End of Part 2 — Advanced System Design Problems